

```
<< " at line " << __LINE__ << endl
<< "          Compiled on " << __DATE__
<< " at " << __TIME__ << endl
<< "          Word read was \"" << word
<< "\"": Length too short" << endl;
```

如果我们给程序提供了一个长度小于 threshold 的 string 对象, 将得到下面的错误消息:

```
Error:wdebug.cc : in function main at line 27
      Compiled on Jul 11 2012 at 20:50:03
      Word read was "foo": Length too short
```

6.5.3 节练习

练习 6.47: 改写 6.3.2 节 (第 205 页) 练习中使用递归输出 vector 内容的程序, 使其有条件地输出与执行过程有关的信息。例如, 每次调用时输出 vector 对象的大小。分别在打开和关闭调试器的情况下编译并执行这个程序。

练习 6.48: 说明下面这个循环的含义, 它对 assert 的使用合理吗?

```
string s;
while (cin >> s && s != sought) { } // 空函数体
assert(cin);
```

6.6 函数匹配



在大多数情况下, 我们容易确定某次调用应该选用哪个重载函数。然而, 当几个重载函数的形参数量相等以及某些形参的类型可以由其他类型转换得来时, 这项工作就不那么容易了。以下面这组函数及其调用为例:

```
void f();
void f(int);
void f(int, int);
void f(double, double = 3.14);
f(5.6);          // 调用 void f(double, double)
```

确定候选函数和可行函数

◀ 243

函数匹配的第一步是选定本次调用对应的重载函数集, 集合中的函数称为**候选函数** (candidate function)。候选函数具备两个特征: 一是与被调用的函数同名, 二是其声明在调用点可见。在这个例子中, 有 4 个名为 f 的候选函数。

第二步考察本次调用提供的实参, 然后从候选函数中选出能被这组实参调用的函数, 这些新选出的函数称为**可行函数** (viable function)。可行函数也有两个特征: 一是其形参数量与本次调用提供的实参数量相等, 二是每个实参的类型与对应的形参类型相同, 或者能转换成形参的类型。

我们能根据实参的数量从候选函数中排除掉两个。不使用形参的函数和使用两个 int 形参的函数显然都不适合本次调用, 这是因为我们的调用只提供了一个实参, 而它们分别有 0 个和两个形参。

使用一个 int 形参的函数和使用两个 double 形参的函数是可行的, 它们都能用一

个实参调用。其中最后那个函数本应该接受两个 `double` 值，但是因为它含有一个默认实参，所以只用一个实参也能调用它。



如果函数含有默认实参（参见 6.5.1 节，第 211 页），则我们在调用该函数时传入的实参数量可能少于它实际使用的实参数量。

在使用实参数量初步判别的候选函数后，接下来考察实参的类型是否与形参匹配。和一般的函数调用类似，实参与形参匹配的含义可能是它们具有相同的类型，也可能是实参类型和形参类型满足转换规则。在上面的例子中，剩下的两个函数都是可行的：

- `f(int)` 是可行的，因为实参类型 `double` 能转换成形参类型 `int`。
- `f(double, double)` 是可行的，因为它的第二个形参提供了默认值，而第一个形参的类型正好是 `double`，与函数使用的实参类型完全一致。

244



如果没找到可行函数，编译器将报告无匹配函数的错误。

寻找最佳匹配（如果有的话）

函数匹配的第三步是从可行函数中选择与本次调用最匹配的函数。在这一过程中，逐一检查函数调用提供的实参，寻找形参类型与实参类型最匹配的那个可行函数。下一节将介绍“最匹配”的细节，它的基本思想是，实参类型与形参类型越接近，它们匹配得越好。

在我们的例子中，调用只提供了一个（显式的）实参，它的类型是 `double`。如果调用 `f(int)`，实参将不得不从 `double` 转换成 `int`。另一个可行函数 `f(double, double)` 则与实参精确匹配。精确匹配比需要类型转换的匹配更好，因此，编译器把 `f(5.6)` 解析成对含有两个 `double` 形参的函数的调用，并使用默认值填补我们未提供的第二个实参。

含有多个形参的函数匹配

当实参的数量有两个或更多时，函数匹配就比较复杂了。对于前面那些名为 `f` 的函数，我们来分析如下的调用会发生什么情况：

```
(42, 2.56);
```

选择可行函数的方法和只有一个实参时一样，编译器选择那些形参数量满足要求且实参类型和形参类型能够匹配的函数。此例中，可行函数包括 `f(int, int)` 和 `f(double, double)`。接下来，编译器依次检查每个实参以确定哪个函数是最佳匹配。如果有且只有一个函数满足下列条件，则匹配成功：

- 该函数每个实参的匹配都不劣于其他可行函数需要的匹配。
- 至少有一个实参的匹配优于其他可行函数提供的匹配。

如果在检查了所有实参之后没有任何一个函数脱颖而出，则该调用是错误的。编译器将报告二义性调用的信息。

在上面的调用中，只考虑第一个实参时我们发现函数 `f(int, int)` 能精确匹配；要想匹配第二个函数，`int` 类型的实参必须转换成 `double` 类型。显然需要内置类型转换的匹配劣于精确匹配，因此仅就第一个实参来说，`f(int, int)` 比 `f(double, double)` 更好。

接着考虑第二个实参 2.56, 此时 `f(double, double)` 是精确匹配; 要想调用 `f(int, int)` 必须将 2.56 从 `double` 类型转换成 `int` 类型。因此仅就第二个实参来说, `f(double, double)` 更好。

编译器最终将因为这个调用具有二义性而拒绝其请求: 因为每个可行函数各自在一个实参上实现了更好的匹配, 从整体上无法判断孰优孰劣。看起来我们似乎可以通过强制类型转换 (参见 4.11.3 节, 第 144 页) 其中的一个实参来实现函数的匹配, 但是在设计良好的系统中, 不应该对实参进行强制类型转换。



调用重载函数时应尽量避免强制类型转换。如果在实际应用中确实需要强制类型转换, 则说明我们设计的形参集合不合理。

6.6 节练习

练习 6.49: 什么是候选函数? 什么是可行函数?

练习 6.50: 已知有第 217 页对函数 `f` 的声明, 对于下面的每一个调用列出可行函数。其中哪个函数是最佳匹配? 如果调用不合法, 是因为没有可匹配的函数还是因为调用具有二义性?

(a) `f(2.56, 42)` (b) `f(42)` (c) `f(42, 0)` (d) `f(2.56, 3.14)`

练习 6.51: 编写函数 `f` 的 4 个版本, 令其各输出一条可以区分的信息。验证上一个练习的答案, 如果你回答错了, 反复研究本节的内容直到你弄清自己错在何处。

6.6.1 实参类型转换



为了确定最佳匹配, 编译器将实参类型到形参类型的转换划分成几个等级, 具体排序如下所示:

1. 精确匹配, 包括以下情况:
 - 实参类型和形参类型相同。
 - 实参从数组类型或函数类型转换成对应的指针类型 (参见 6.7 节, 第 221 页, 将介绍函数指针)。
 - 向实参添加顶层 `const` 或者从实参中删除顶层 `const`。
2. 通过 `const` 转换实现的匹配 (参见 4.11.2 节, 第 143 页)。
3. 通过类型提升实现的匹配 (参见 4.11.1 节, 第 142 页)。
4. 通过算术类型转换 (参见 4.11.1 节, 第 142 页) 或指针转换 (参见 4.11.2 节, 第 143 页) 实现的匹配。
5. 通过类类型转换实现的匹配 (参见 14.9 节, 第 514 页, 将详细介绍这种转换)。

需要类型提升和算术类型转换的匹配



内置类型的提升和转换可能在函数匹配时产生意想不到的结果, 但幸运的是, 在设计良好的系统中函数很少会含有与下面例子类似的形参。

分析函数调用前, 我们应该知道小整型一般都会提升到 `int` 类型或更大的整数类型。

假设有两个函数，一个接受 `int`、另一个接受 `short`，则只有当调用提供的是 `short` 类型的值时才会选择 `short` 版本的函数。有时候，即使实参是一个很小的整数值，也会直接将它提升成 `int` 类型；此时使用 `short` 版本反而会导致类型转换：



```
void ff(int);
void ff(short);
ff('a');           // char 提升成 int; 调用 f(int)
```

所有算术类型转换的级别都一样。例如，从 `int` 向 `unsigned int` 的转换并不比从 `int` 向 `double` 的转换级别高。举个具体点的例子，考虑

```
void manip(long);
void manip(float);
manip(3.14);       // 错误：二义性调用
```

字面值 `3.14` 的类型是 `double`，它既能转换成 `long` 也能转换成 `float`。因为存在两种可能的算术类型转换，所以该调用具有二义性。

函数匹配和 `const` 实参

如果重载函数的区别在于它们的引用类型的形参是否引用了 `const`，或者指针类型的形参是否指向 `const`，则当调用发生时编译器通过实参是否是常量来决定选择哪个函数：

```
Record lookup(Account&);           // 函数的参数是 Account 的引用
Record lookup(const Account&);      // 函数的参数是一个常量引用
const Account a;
Account b;

lookup(a);                         // 调用 lookup(const Account&)
lookup(b);                         // 调用 lookup(Account&)
```

在第一个调用中，我们传入的是 `const` 对象 `a`。因为不能把普通引用绑定到 `const` 对象上，所以此例中唯一可行的函数是以常量引用作为形参的那个函数，并且调用该函数与实参 `a` 精确匹配。

在第二个调用中，我们传入的是非常量对象 `b`。对于这个调用来说，两个函数都是可行的，因为我们既可以使用 `b` 初始化常量引用也可以使用它初始化非常量引用。然而，用非常量对象初始化常量引用需要类型转换，接受非常量形参的版本则与 `b` 精确匹配。因此，应该选用非常量版本的函数。

247

指针类型的形参也类似。如果两个函数的唯一区别是它的指针形参指向常量或非常量，则编译器能通过实参是否是常量决定选用哪个函数：如果实参是指向常量的指针，调用形参是 `const*` 的函数；如果实参是指向非常量的指针，调用形参是普通指针的函数。

6.6.1 节练习

练习 6.52：已知有如下声明，

```
void manip(int, int);
double dobj;
```

请指出下列调用中每个类型转换的等级（参见 6.6.1 节，第 219 页）。

(a) `manip('a', 'z');` (b) `manip(55.4, dobj);`

练习 6.53：说明下列每组声明中的第二条语句会产生什么影响，并指出哪些不合法（如

果有的话)。

```
(a) int calc(int&, int&);
    int calc(const int&, const int&);
(b) int calc(char*, char*);
    int calc(const char*, const char*);
(c) int calc(char*, char*);
    int calc(char* const, char* const);
```



6.7 函数指针

函数指针指向的是函数而非对象。和其他指针一样，函数指针指向某种特定类型。函数的类型由它的返回类型和形参类型共同决定，与函数名无关。例如：

```
// 比较两个 string 对象的长度
bool lengthCompare(const string &, const string &);
```

该函数的类型是 `bool(const string&, const string&)`。要想声明一个可以指向该函数的指针，只需要用指针替换函数名即可：

```
// pf 指向一个函数，该函数的参数是两个 const string 的引用，返回值是 bool 类型
bool (*pf)(const string &, const string &); // 未初始化
```

从我们声明的名字开始观察，pf 前面有个*，因此 pf 是指针；右侧是形参列表，表示 pf 指向的是函数；再观察左侧，发现函数的返回类型是布尔值。因此，pf 就是一个指向函数的指针，其中该函数的参数是两个 const string 的引用，返回值是 bool 类型。



*pf 两端的括号必不可少。如果不写这对括号，则 pf 是一个返回值为 bool 指针的函数：

```
// 声明一个名为 pf 的函数，该函数返回 bool*
bool *pf(const string &, const string &);
```

◀ 248

使用函数指针

当我们把函数名作为一个值使用时，该函数自动地转换成指针。例如，按照如下形式我们可以将 lengthCompare 的地址赋给 pf：

```
pf = lengthCompare;           // pf 指向名为 lengthCompare 的函数
pf = &lengthCompare;         // 等价的赋值语句：取地址符是可选的
```

此外，我们还能直接使用指向函数的指针调用该函数，无须提前解引用指针：

```
bool b1 = pf("hello", "goodbye");           // 调用 lengthCompare 函数
bool b2 = (*pf)("hello", "goodbye");         // 一个等价的调用
bool b3 = lengthCompare("hello", "goodbye"); // 另一个等价的调用
```

在指向不同函数类型的指针间不存在转换规则。但是和往常一样，我们可以为函数指针赋一个 nullptr（参见 2.3.2 节，第 48 页）或者值为 0 的整型常量表达式，表示该指针没有指向任何一个函数：

```
string::size_type sumLength(const string&, const string&);
bool cstringCompare(const char*, const char*);
pf = 0;                               // 正确：pf 不指向任何函数
pf = sumLength;                        // 错误：返回类型不匹配
```